



Universidad Nacional Autónoma de México

Computer Engineering

Compilers

Final Project C-Pure Compiler

Team 7

Students:

- 321189536
- 321184191
- 117002933
- 321134567
- 321335630

Group:

5

Semester:

2026-II

Mexico, CDMX. April 28, 2026

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Motivation	2
1.3	Objectives	3
2	Theoretical Framework	3
3	Body	7
3.1	Lexical Analyzer	7
3.2	Parser	10
3.3	FIRST and FOLLOW Sets	15
3.4	LL(1) Parsing Table	18
3.5	Proof that the grammar is LL(1)	18
3.6	Parser implementation	21
3.7	Abstract Syntax Tree (AST)	22
3.8	Semantic Analyzer	24
3.9	Why Top-Down and not Bottom-Up	30
3.10	Three-Address Code Generation	30
3.11	Assembler Code Generation	35
3.12	Use-Case Based GUI Validation	39

1. Introduction

1.1 Problem Statement

The translation and compilation process is traditionally divided into a sequence of analyses that progressively transform source code from raw text into a structured and lower-level representation.

A compiler cannot treat a program as plain text. It must first identify the atomic components of the language, verify that these components form valid syntactic structures, and ensure that the resulting program complies with the semantic rules established by the language.

In the context of developing a complete compiler for a subset of C, the challenge extends beyond preliminary static analysis. The system must consolidate its front-end and back-end by integrating lexical analysis, LL(1) predictive parsing, Syntax-Directed Translation (SDT), Abstract Syntax Tree (AST) construction, intermediate-code generation and assembler output generation.

This project addresses that problem through the implementation of a C-Pure compiler pipeline. The system receives source code, classifies it into tokens, validates the token stream using an LL(1) grammar and parsing table, applies semantic rules through an SDT scheme, builds an AST as an intermediate representation, generates Three-Address Code (TAC), and finally translates that intermediate representation into low-level assembler instructions.

1.2 Motivation

Syntactic and semantic analysis constitute fundamental stages in compiler design, as they ensure that the source code is structurally coherent and free of ambiguities. Implementing a predictive parser allows a deeper understanding of how programming languages are formally described and how their grammatical constructs can be recognized using deterministic procedures.

Furthermore, incorporating a Syntax-Directed Translation (SDT) scheme enables the integration of semantic validation into the analysis process, allowing the detection of errors such as undeclared variables, invalid function calls, type mismatches, and incorrect use of language constructs. This stage also facilitates the construction of structured representations such as Abstract Syntax Trees (ASTs), which are essential for subsequent phases of compilation.

Finally, the generation of intermediate code and assembler instructions connects the front-end stages with the back-end of the compiler. This allows the project to demonstrate how a validated AST can be transformed into a lower-level representation, showing the complete transition from source code to generated instructions.

1.3 Objectives

- Implement a lexical analyzer capable of reading C-Pure source code from a file or input string, removing comments, recognizing valid lexemes through regular expressions, classifying them into token categories and reporting unknown tokens as lexical errors.
- Define an unambiguous, left-factored, left-recursion-free context-free grammar for a subset of C covering declarations, assignments, control flow, function definitions, parameters, calls with arguments, and arithmetic, comparison and logical expressions with operator precedence.
- Prove formally that the grammar is LL(1) by computing its FIRST and FOLLOW sets and showing that no production conflicts arise at any decision point.
- Implement a Top-Down predictive parser that consumes the token stream produced by the lexer, uses an explicit control stack, and produces an attributed AST.
- Integrate semantic actions through an SDT scheme to detect redeclaration of identifiers, use of undeclared identifiers, invalid function calls, type incompatibility in declarations, assignments, expressions and **return** statements, and invalid use of control-flow constructs.
- Generate intermediate code using Three-Address Code (TAC), representing expressions and control flow through temporaries, labels and jumps.
- Translate the generated TAC into assembler instructions that illustrate the final code-generation stage of the compiler.

2. Theoretical Framework

Lexical Analyzer

A lexical analyzer is a program that converts the input text into tokens and classifies these tokens into predefined categories [1]. The lexical analyzer plays a crucial role in the analysis phase, since it provides the tokens to the parser to facilitate syntax analysis. In this project, the lexical analyzer reads a string or file containing C code and identifies the tokens that compose the program. [2]

Token

A token is a sequence of characters grouped into a single entity. Each token represents a set of character sequences that convey a specific meaning within the programming language. Tokens are the basic units that allow the compiler to understand the structure of the source code [2].

Token Classification

In this project, the lexical analyzer classifies tokens into the following categories:

- Keywords
- Identifiers
- Constants
- Literals
- Operators
- Punctuation
- Unknown tokens [3]

Keywords correspond to reserved words of the C language. Identifiers represent the names assigned to variables, functions, or other program elements. Constants represent numeric or character values that do not change during program execution. Literals represent sequences of characters enclosed in quotation marks. Operators represent arithmetic, logical, relational, or assignment operations. Punctuation symbols are characters used to structure the syntax of the program. Unknown tokens represent sequences of characters that cannot be classified into any valid token category [4].

Lexical Errors

A lexical error occurs when the lexical analyzer encounters a sequence of characters that cannot be classified as a valid token according to the lexical rules of the language. These errors may appear due to illegal characters, malformed constants or invalid sequences that do not correspond to any token pattern. In this project, such cases are classified as *Unknown tokens*, allowing the compiler pipeline to stop before syntactic analysis when the input contains invalid lexical elements. [5]

Regular Expressions

Regular expressions are a formal notation used to describe patterns within strings. In lexical analysis, they define the structure that valid tokens must follow. Each token category can be represented by a regular expression that specifies the allowed combination of characters. For example, identifiers in C-Pure follow the pattern `[a-zA-Z_][a-zA-Z0-9_]*`, which indicates that an identifier must begin with a letter or underscore and may be followed by letters, digits or underscores. [6]

Finite Automata

A finite automaton is an abstract machine used to recognize patterns by processing symbols one at a time and transitioning between a finite number of states. Since the lexical structure of programming languages can be described using regular languages, finite automata provide the theoretical foundation for recognizing keywords, identifiers, constants, literals, operators and punctuation symbols. [7]

Parser

A parser, or syntactic analyzer, is the second phase of a compiler. It receives the token stream produced by the lexer and decides whether the sequence of tokens forms a sentence in the language defined by a given context-free grammar. The output of a successful parse is a derivation or, equivalently, a parse tree that exposes the hierarchical structure of the program. Parsers are classified into *Top-Down* parsers, which build the tree from the start symbol toward the input, and *Bottom-Up* parsers, which build it from the input toward the start symbol [8].

Top-Down Parsing (LL(1))

A Top-Down parser builds the parse tree from the root down to the leaves, reading the input from left to right and producing a *leftmost derivation*. An LL(1) parser looks one token ahead and decides which production to apply using a predictive parsing table built from the FIRST and FOLLOW sets [9].

FIRST and FOLLOW sets

For a grammar symbol X , $\text{FIRST}(X)$ is the set of terminals that can begin some string derived from X ; if X can derive the empty string, ε is also included. For a non-terminal A , $\text{FOLLOW}(A)$ is the set of terminals that may appear immediately to the right of A in some sentential form; the end-of-input marker $\$$ belongs to $\text{FOLLOW}(S)$ for the start symbol S .

A grammar is **LL(1)** if, for every pair of productions $A \rightarrow \alpha$ and $A \rightarrow \beta$:

- $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$, and
- whenever $\varepsilon \in \text{FIRST}(\alpha)$, also $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$ [10].

Syntax-Directed Translation (SDT)

A Syntax-Directed Translation is a context-free grammar enriched with *semantic rules* associated with its productions. Each grammar symbol carries one or more *attributes*; the rules describe how those attributes are computed during parsing. Attributes can be *synthesized* (computed from the children of a node) or *inherited* (computed from the parent or siblings). Typical uses of an SDT include type checking, symbol-table construction, expression evaluation and the generation of intermediate code.

In our project the SDT is woven directly into the parsing process: semantic actions validate declarations, scopes, function usage and type compatibility while the AST is built and prepared for later code-generation phases [11].

Three-Address Code (TAC)

Three-Address Code is an intermediate representation in which each instruction contains at most one operator. It is useful for representing assignments, expressions, conditions and jumps in a linear form that is easier to translate into lower-level code. [12]

Assembler Instructions

Assembler instructions represent a low-level output generated from the intermediate code. In this project, these instructions illustrate how TAC can be translated into operations such as data movement, arithmetic operations, comparisons and jumps. [13]

3. Body

3.1 Lexical Analyzer

The lexical analyzer constitutes the first operational phase of the C-Pure compiler pipeline. Its main responsibility is to transform the source program from a raw sequence of characters into an ordered stream of tokens that can be consumed by the syntactic analyzer. This stage is necessary because the parser does not work directly with characters; it works with classified units such as keywords, identifiers, operators, constants, literals and punctuation symbols.

The lexer reads the source code line by line. Before token recognition, comments are removed so that they do not interfere with the syntactic structure of the program. After this preprocessing step, the remaining character sequences are compared against the lexical patterns defined for each token category. When a valid pattern is matched, the corresponding token is generated with its type, lexeme value, line number and column number. This positional information is useful for later error reporting in the compiler pipeline.

3.1.1 Token recognition strategy

The recognition process follows a priority-based strategy. Reserved words are treated as keywords, while user-defined names that satisfy the identifier pattern are classified as identifiers. Numeric values are classified as constants; string or character sequences enclosed in quotation marks are classified as literals; and structural symbols are classified as punctuation. Operators are recognized according to their longest valid match, which is important for distinguishing one-character operators from compound operators such as `==`, `!=`, `>=`, `<=`, `&&`, `||`, `++` and `-`.

The lexical analyzer supports the following token categories:

- **Keywords:** reserved words of the language, such as `int`, `float`, `double`, `char`, `void`, `if`, `else`, `while`, `do`, `for` and `return`.
- **Identifiers:** names assigned to variables, functions and parameters.
- **Constants:** numeric values such as integers, floating point values, hexadecimal values and scientific notation forms.
- **Literals:** character or string values enclosed in quotation marks.
- **Operators:** arithmetic, assignment, relational and logical operators.
- **Punctuation:** delimiters and structural symbols such as parentheses, braces, brackets, commas and semicolons.

- **Unknown tokens:** invalid sequences that do not match any valid lexical pattern.

3.1.2 Regular expressions used by the lexer

The lexical rules are implemented through regular expressions. These expressions describe the valid structure of each token category and allow the lexer to classify lexemes systematically.

Listing 1: Regular expression for identifiers

Identifiers —
`[a-zA-Z_][a-zA-Z0-9_]*`

This expression indicates that an identifier must start with a letter or underscore and may be followed by any number of letters, digits or underscores. Examples of valid identifiers are `x`, `main`, `counter1` and `_temp`.

Listing 2: Regular expression for operators

Operators —
`>=>|<=<|\+=|-=|*=|/=|%=|==|!=|>|<|&&|\|\|\|\+|\+|--|
<<|>>|&=|\|=|\^=|=|>|<|!|~|\+|-|*|/|%|&|\|\|\|^\|?|:`

This expression recognizes assignment, arithmetic, relational, logical and compound operators. The ordering of the alternatives is important because multi-character operators must be matched before their single-character prefixes.

Listing 3: Regular expression for punctuation symbols

Punctuation —
`\.\.\.\.|[( ) \ [ \ ] { } ; , . : #`

This expression identifies the punctuation symbols used to structure C-Pure programs. Parentheses delimit expressions and parameter lists, braces delimit statement blocks, commas separate parameters or arguments, and semicolons terminate statements.

Listing 4: Regular expression for numeric constants

Constants —
`0[xX][0-9a-fA-F]+ | 0[0-7]+ | (0|[1-9][0-9]*) |
([0-9]+\.[0-9]+([eE][+-]?[0-9]+)?) |
(\.[0-9]+([eE][+-]?[0-9]+)?) |
([0-9]+[eE][+-]?[0-9]+)`

This expression allows the recognition of decimal, octal, hexadecimal, floating point and scientific notation constants. In the semantic rules of the project, numeric constants are treated as integer-compatible values unless the implementation infers a more specific numeric type.

Listing 5: Regular expression for string literals

Literals

```
"([^\\"\\]|\\.)*"
```

This expression recognizes string values enclosed in double quotation marks. The inner expression allows regular characters and escaped sequences. Character literals enclosed in single quotation marks are also treated as literal values by the lexical phase and are later interpreted semantically according to the literal conventions of the language.

Listing 6: Reserved words recognized by the lexer

Keywords

```
auto | break | case | char | const | continue | default | do | double | else |  
    enum |  
extern | float | for | goto | if | inline | int | long | register | restrict |  
return | short | signed | sizeof | static | struct | switch | typedef |  
union | unsigned | void | volatile | while
```

The keyword list contains reserved words from the C language. However, lexical recognition and syntactic acceptance are different processes. A word can be recognized as a valid keyword by the lexer, but it will only be accepted by the parser if it belongs to one of the productions defined in the C-Pure grammar.

3.1.3 Comment and whitespace handling

Whitespace characters such as spaces, tabs and line breaks are used only as separators between lexemes and are not emitted as tokens. This allows the parser to receive only meaningful lexical units. The analyzer also removes single-line and block comments before token classification, ensuring that comments do not produce invalid tokens or interfere with the syntactic structure of the program.

Preserving line information is important even when comments are removed. Therefore, the lexical analyzer keeps enough positional information to report where each token or lexical error appears. This behavior supports later stages of the compiler, since syntax and semantic error messages can refer to specific lines and columns.

3.1.4 Lexical error detection

When the analyzer finds a sequence that does not match any valid token pattern, that sequence is classified as an **Unknown** token. This mechanism allows the compiler to detect lexical errors before attempting syntactic analysis. If unknown tokens are present, the compilation pipeline should stop at the lexical phase, because the parser cannot reliably process invalid lexical units.

For example, in an expression such as:

Listing 7: Example of lexical error

```
int x = 10 @ 5;
```

the symbol `@` is not part of the supported C-Pure lexical structure. Therefore, the lexer classifies it as an unknown token and reports it as a lexical error.

3.1.5 Output of the lexical phase

The output of the lexical analyzer is an ordered token stream. Each token stores at least the following information:

- **Token type:** lexical category of the token.
- **Value:** original lexeme found in the source code.
- **Line:** line number where the lexeme was found.
- **Column:** column where the lexeme starts.

A representative tokenization of the instruction `int x = 10;` is shown below:

Listing 8: Representative lexical output

```
[int]    -> Keywords
[x]      -> Identifiers
[=]      -> Operators
[10]     -> Constants
[;]      -> Punctuation
```

This token stream becomes the formal input of the LL(1) parser. In this way, the lexical analyzer acts as the bridge between raw source code and the structured syntactic analysis performed by the rest of the compiler.

3.2 Parser

The development of the parser was carried out in several stages. First, a context-free grammar based on a subset of the C language was defined. This grammar was structured through left factoring and elimination of left recursion to make it compatible with LL(1) predictive parsing. Subsequently, the FIRST and FOLLOW sets were computed in order to verify the absence of ambiguities and ensure that the grammar was suitable for the construction of a deterministic predictive parser.

Based on this theoretical foundation, the syntactic analyzer was implemented using a Top-Down LL(1) approach. The parser consumes the token stream produced by the lexical analyzer and uses a predictive parsing table to decide which production must be applied according to the current non-terminal and lookahead token. During this

process, semantic actions and AST construction are also integrated to prepare the program for semantic validation and later code-generation stages.

3.2.1 Context Free Grammar (CFG)

The syntactic structure of the language is defined using a Context-Free Grammar (CFG), derived from a simplified subset of the C programming language. This grammar was specifically designed to be compatible with LL(1) predictive parsing, ensuring the absence of ambiguities during the syntactic analysis process.

The grammar allows the declaration of global and local variables, as well as their optional initialization. It supports basic data types such as `int`, `float`, `double`, `char`, and `void`. Additionally, it allows the definition of functions with parameter lists, the use of function calls with arguments, and the construction of statement blocks.

The language also supports conditional control structures such as `if`, `if-else`, and `else-if`, as well as loop structures such as `while`, `do-while`, and `for`. It includes return statements, assignments, increment and decrement statements, and arithmetic, relational, equality and logical expressions with precedence levels defined through the grammar hierarchy.

The grammar is composed of four fundamental components: terminals, non-terminals, production rules, and the start symbol.

Terminals represent the basic lexical units of the language, provided by the lexical analyzer. They include reserved keywords such as `int`, `float`, `double`, `char`, `void`, `if`, `else`, `while`, `do`, `for`, and `return`, as well as identifiers, constants, literals, arithmetic operators, relational operators, logical operators, and punctuation symbols. Terminals represent the leaves of the parse tree and cannot be further decomposed.

Non-terminals define the syntactic structure of the language. They represent higher-level constructs such as programs, global and local declarations, parameters, arguments, statements, expressions, function definitions, control structures and loop structures. These abstract symbols are expanded during syntactic analysis through the application of production rules.

Production rules describe how non-terminals can be replaced by combinations of terminals and other non-terminals. These rules define the valid structure of programs in the language, including variable declarations, function definitions, function calls, conditional statements, loop statements, assignments, return statements and expressions with different levels of precedence and associativity.

To enable predictive parsing, the grammar was transformed by eliminating left recursion and applying left factoring, ensuring that each non-terminal can be parsed deterministically using a single lookahead token.

Start symbol is defined as `PROGRAM`, which represents the entry point of the language.

From this symbol, all valid programs are derived through successive applications of production rules.

Language restrictions (derived from the grammar):

- Arrays, pointers, structures, unions and dynamic memory management are not supported.
- Standard input/output operations are not part of the core grammar.
- Only constructs explicitly defined by the grammar are considered valid during syntactic analysis.
- The generated assembler output is an educational low-level representation and not a fully executable binary program.

These restrictions are intentional to maintain the grammar in LL(1) form, eliminating ambiguities and enabling the construction of an efficient predictive parser. This also facilitates the integration with semantic analysis, AST construction and code generation.

Listing 9: C-Pure context-free grammar

PROGRAM	→ GLOBAL PROGRAM
PROGRAM	→ epsilon
GLOBAL	→ TYPE id GLOBAL_REST
TYPE	→ int
TYPE	→ float
TYPE	→ double
TYPE	→ char
TYPE	→ void
GLOBAL_REST	→ (PARAMS) { STMT_LIST }
GLOBAL_REST	→ OPT_ASSIGN ;
PARAMS	→ TYPE id PARAMS_REST
PARAMS	→ epsilon
PARAMS_REST	→ , TYPE id PARAMS_REST
PARAMS_REST	→ epsilon
OPT_ASSIGN	→ = E
OPT_ASSIGN	→ epsilon

STMT_LIST	→	STMT STMT_LIST
STMT_LIST	→	epsilon
STMT	→	IF_STMT
STMT	→	WHILE_STMT
STMT	→	DO_WHILE_STMT
STMT	→	FOR_STMT
STMT	→	return OPT_E ;
STMT	→	TYPE id OPT_ASSIGN ;
STMT	→	id STMT_ID_REST
OPT_E	→	E
OPT_E	→	epsilon
STMT_ID_REST	→	= E ;
STMT_ID_REST	→	(ARGS) ;
STMT_ID_REST	→	++ ;
STMT_ID_REST	→	-- ;
ARGS	→	E ARGS_REST
ARGS	→	epsilon
ARGS_REST	→	, E ARGS_REST
ARGS_REST	→	epsilon
IF_STMT	→	if (E) { STMT_LIST } ELSE_PART
ELSE_PART	→	else ELSE_CHOICE
ELSE_PART	→	epsilon
ELSE_CHOICE	→	{ STMT_LIST }
ELSE_CHOICE	→	IF_STMT
WHILE_STMT	→	while (E) { STMT_LIST }
DO_WHILE_STMT	→	do { STMT_LIST } while (E) ;
FOR_STMT	→	for (FOR_INIT E ; FOR_UPD) { STMT_LIST }
FOR_INIT	→	TYPE id = E ;

```

FOR_INIT      → id = E ;
FOR_INIT      → ;

FOR_UPD       → id FOR_UPD_REST
FOR_UPD       → epsilon

FOR_UPD_REST  → ++
FOR_UPD_REST  → --
FOR_UPD_REST  → = E

E             → LOGIC_OR

LOGIC_OR      → LOGIC_AND LOGIC_OR_PRIME
LOGIC_OR_PRIME → || LOGIC_AND LOGIC_OR_PRIME
LOGIC_OR_PRIME → epsilon

LOGIC_AND     → EQUALITY LOGIC_AND_PRIME
LOGIC_AND_PRIME → && EQUALITY LOGIC_AND_PRIME
LOGIC_AND_PRIME → epsilon

EQUALITY      → COMPARISON EQUALITY_PRIME
EQUALITY_PRIME → == COMPARISON EQUALITY_PRIME
EQUALITY_PRIME → != COMPARISON EQUALITY_PRIME
EQUALITY_PRIME → epsilon

COMPARISON    → TERM COMPARISON_PRIME
COMPARISON_PRIME → > TERM COMPARISON_PRIME
COMPARISON_PRIME → < TERM COMPARISON_PRIME
COMPARISON_PRIME → >= TERM COMPARISON_PRIME
COMPARISON_PRIME → <= TERM COMPARISON_PRIME
COMPARISON_PRIME → epsilon

TERM          → FACTOR TERM_PRIME
TERM_PRIME    → + FACTOR TERM_PRIME
TERM_PRIME    → - FACTOR TERM_PRIME
TERM_PRIME    → epsilon

FACTOR        → UNARY FACTOR_PRIME
FACTOR_PRIME  → * UNARY FACTOR_PRIME
FACTOR_PRIME  → / UNARY FACTOR_PRIME
FACTOR_PRIME  → % UNARY FACTOR_PRIME

```

FACTOR_PRIME	→	epsilon
UNARY	→	! UNARY
UNARY	→	- UNARY
UNARY	→	PRIMARY
PRIMARY	→	id
PRIMARY	→	constant
PRIMARY	→	literal
PRIMARY	→	(E)

3.3 FIRST and FOLLOW Sets

To construct the LL(1) predictive syntactic analyzer, the FIRST and FOLLOW sets were calculated for each non-terminal in the grammar. These sets are fundamental for ensuring deterministic production selection during the parsing process, thereby eliminating ambiguities and potential conflicts when choosing derivation rules.

The **FIRST** set of a non-terminal comprises all terminal symbols that can appear at the beginning of strings derived from that non-terminal, including the empty string (ϵ) for nullable productions.

Conversely, the **FOLLOW** set contains the set of terminal symbols that can appear immediately to the right of a non-terminal in any valid derivation. The end-of-input marker (\$) is, by definition, included in the FOLLOW set of the start symbol, **PROGRAM**.

FIRST Sets

Non-Terminal	FIRST Set
PROGRAM	{ char, double, float, int, void, ϵ }
GLOBAL, TYPE	{ char, double, float, int, void }
GLOBAL_REST	{ (, ;, = }
PARAMS	{ char, double, float, int, void, ϵ }
PARAMS_REST, ARGS_REST	{ „ ϵ }
OPT_ASSIGN	{ =, ϵ }
STMT_LIST	{ char, do, double, float, for, id, if, int, return, void, while, ϵ }
STMT	{ char, do, double, float, for, id, if, int, return, void, while }
STMT_ID_REST	{ (, ++, -, = }
OPT_E, ARGS	{ !, (, -, constant, id, literal, ϵ }
IF_STMT	{ if }
ELSE_PART	{ else, ϵ }
ELSE_CHOICE	{ if, { }
WHILE_STMT	{ while }
DO_WHILE_STMT	{ do }
FOR_STMT	{ for }
FOR_INIT	{ ;, char, double, float, id, int, void }
FOR_UPD	{ id, ϵ }
FOR_UPD_REST	{ ++, -, = }
E, LOGIC_OR, LOGIC_AND, EQUALITY	{ !, (, -, constant, id, literal }
COMPARISON, TERM, FACTOR, UNARY	{ !, (, -, constant, id, literal }
PRIMARY	{ (, constant, id, literal }
LOGIC_OR_PRIME	{ , ϵ }
LOGIC_AND_PRIME	{ &&, ϵ }
EQUALITY_PRIME	{ !=, ==, ϵ }
COMPARISON_PRIME	{ <, <=, >, >=, ϵ }
TERM_PRIME	{ +, -, ϵ }
FACTOR_PRIME	{ %, *, /, ϵ }

Table 1: FIRST sets for the C-Pure grammar.

FOLLOW Sets

Non-Terminal	FOLLOW Set
PROGRAM	{ \$ }
GLOBAL, GLOBAL_REST	{ \$, char, double, float, int, void }
TYPE	{ id }
PARAMS, PARAMS_REST	{) }
OPT_ASSIGN, OPT_E	{ ; }
STMT_LIST	{ } }
STMT, STMT_ID_REST, IF_STMT, ELSE_PART, ELSE_CHOICE	{ char, do, double, float, for, id, if, int, return, void, while, } }
WHILE_STMT, DO_WHILE_STMT, FOR_STMT	{ char, do, double, float, for, id, if, int, return, void, while, } }
ARGS, ARGS_REST	{) }
FOR_INIT	{ !, (, -, constant, id, literal }
FOR_UPD, FOR_UPD_REST	{) }
E, LOGIC_OR, LOGIC_OR_PRIME	{), „ ; }
LOGIC_AND, LOGIC_AND_PRIME	{ &&,), „ ;, }
EQUALITY, EQUALITY_PRIME	{ &&,), „ ;, }
COMPARISON, COMPARISON_PRIME	{ !=, &&,), „ ;, ==, }
TERM, TERM_PRIME	{ !=, &&,), „ ;, <, <=, ==, >, >=, }
FACTOR, FACTOR_PRIME	{ !=, &&,), +, „ -, ;, <, <=, ==, >, >=, }
UNARY, PRIMARY	{ !=, %, &&,), *, +, „ -, /, ;, <, <=, ==, >, >=, }

Table 2: FOLLOW sets for the C-Pure grammar.

The calculated FIRST and FOLLOW sets confirm that the grammar is suitable for LL(1) predictive parsing, as there are no selection conflicts between alternative productions. Furthermore, the systematic handling of ϵ within the FIRST sets allows for the correct parsing of optional language constructs. These sets serve as the

mathematical foundation for the parsing table, ensuring deterministic behavior in the parser.

3.4 LL(1) Parsing Table

The LL(1) parsing table is a two-dimensional matrix $M[A, a]$, where A represents a non-terminal symbol and a represents a terminal symbol or the end-of-input marker ($\$$). This table serves as the lookup mechanism for the predictive parser, determining which production rule to apply based on the current non-terminal at the top of the stack and the lookahead token provided by the lexical analyzer.

Construction Methodology

The construction of the parsing table was performed systematically using the previously computed FIRST and FOLLOW sets. For each production rule of the form $A \rightarrow \alpha$, the following procedure was applied:

- **Direct Terminals:** For every terminal $a \in FIRST(\alpha)$, the production $A \rightarrow \alpha$ is inserted into the entry $M[A, a]$.
- **Handling Epsilon (ϵ):** If $\epsilon \in FIRST(\alpha)$, then for every terminal $b \in FOLLOW(A)$, the production $A \rightarrow \alpha$ is added to $M[A, b]$.
- **End of Input:** If $\epsilon \in FIRST(\alpha)$ and $\$$ (end-of-input marker) is in $FOLLOW(A)$, then the production $A \rightarrow \alpha$ is also added to $M[A, \$]$.

Conflict Resolution and Determinism

A key objective during the construction process was to ensure that the grammar remains LL(1)-compliant. This is verified by the fact that no cell in the table contains more than one production rule. The absence of conflicts confirms that:

- For any non-terminal A , the FIRST sets of its alternative productions are pairwise disjoint.
- If a non-terminal can derive ϵ , then its FIRST set does not overlap with its FOLLOW set.

3.5 Proof that the grammar is LL(1)

A grammar is LL(1) if for every pair of alternatives $A \rightarrow \alpha \mid \beta$ the following conditions hold:

- (i) $FIRST(\alpha) \cap FIRST(\beta) = \emptyset$

- (ii) If $\varepsilon \in \text{FIRST}(\alpha)$ then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$

We verify the grammar by analyzing each decision point.

Program structure —

- $\text{PROGRAM} \rightarrow \text{GLOBAL PROGRAM} \mid \varepsilon$:

$\text{FIRST}(\text{GLOBAL PROGRAM}) = \text{FIRST}(\text{GLOBAL}) = \{\text{int}, \text{float}, \text{double}, \text{char}, \text{void}\}$

$\text{FOLLOW}(\text{PROGRAM}) = \{\$ \}$

Therefore:

$$\text{FIRST}(\text{GLOBAL}) \cap \text{FOLLOW}(\text{PROGRAM}) = \emptyset$$

Condition (ii) holds.

Global declarations and function headers —

- $\text{GLOBAL} \rightarrow \text{TYPE id GLOBAL_REST}$

$\text{FIRST}(\text{GLOBAL}) = \text{FIRST}(\text{TYPE}) = \{\text{int}, \text{float}, \text{double}, \text{char}, \text{void}\}$

TYPE alternatives are disjoint because each one begins with a different terminal.

- GLOBAL_REST :

– $(\text{PARAMS}) \{ \text{STMT_LIST} \}$

– $\text{OPT_ASSIGN} ;$

FIRST sets:

$$\{ () \cap \{ =, ;, \varepsilon \} = \emptyset$$

Therefore no conflict exists between function definitions and variable declarations.

Parameters and arguments —

- $\text{PARAMS} \rightarrow \text{TYPE id PARAMS_REST} \mid \varepsilon$

$\text{FIRST}(\text{TYPE id PARAMS_REST}) = \{\text{int}, \text{float}, \text{double}, \text{char}, \text{void}\}$

$\text{FOLLOW}(\text{PARAMS}) = \{ \}$

$$\text{FIRST}(\text{TYPE}) \cap \text{FOLLOW}(\text{PARAMS}) = \emptyset$$

- $\text{ARGS} \rightarrow \text{E ARGS_REST} \mid \varepsilon$

$\text{FIRST}(\text{E}) = \{!, (, -, \text{constant}, \text{id}, \text{literal}\}$

$\text{FOLLOW}(\text{ARGS}) = \{ \}$

$$\text{FIRST}(E) \cap \text{FOLLOW}(\text{ARGS}) = \emptyset$$

Statements —

- STMT has alternatives that begin with distinct terminals:
 - IF_STMT starts with **if**
 - WHILE_STMT starts with **while**
 - DO_WHILE_STMT starts with **do**
 - FOR_STMT starts with **for**
 - return starts with **return**
 - TYPE id starts with TYPE
 - id STMT_ID_REST starts with id

Therefore, the FIRST sets of the statement alternatives are pairwise disjoint.

- STMT_ID_REST has alternatives that begin with =, (, ++, and -. These terminals are distinct, so no FIRST/FIRST conflict exists.

Conditional and loop statements —

- IF_STMT starts with **if**, WHILE_STMT starts with **while**, DO_WHILE_STMT starts with **do**, and FOR_STMT starts with **for**. Since each control-flow production begins with a different reserved word, selection is deterministic.
- $\text{ELSE_PART} \rightarrow \text{else ELSE_CHOICE} \mid \varepsilon$ is resolved through the lookahead token. If the lookahead is **else**, the first production is selected; otherwise, the epsilon production is selected using FOLLOW(ELSE_PART).

Expressions (LL(1) hierarchy) — Each level follows:

$$X \rightarrow YX' \quad , \quad X' \rightarrow op \ Y \ X' \mid \varepsilon$$

- LOGIC_OR_PRIME: {||}
- LOGIC_AND_PRIME: {&&}
- EQUALITY_PRIME: {==, !=}
- COMPARISON_PRIME: {>, <, >=, <=}
- TERM_PRIME: {+, -}
- FACTOR_PRIME: {*, /, %}

All FOLLOW sets exclude their own continuation operators, so:

$$\text{FIRST}(op) \cap \text{FOLLOW}(X') = \emptyset$$

Thus, the expression grammar satisfies the LL(1) conditions.

Unary and primary —

- $\text{UNARY} \rightarrow ! \text{UNARY} \mid - \text{UNARY} \mid \text{PRIMARY}$

FIRST sets:

$$\{!, -\} \cap \text{FIRST}(\text{PRIMARY}) = \emptyset$$

- PRIMARY alternatives begin with `id`, `constant`, `literal`, and `(`, which are all distinct terminals.

Conclusion — All production rules satisfy LL(1) conditions: no FIRST/FIRST or FIRST/FOLLOW conflicts exist. Therefore, the grammar is LL(1) and can be parsed using a predictive Top-Down parser without backtracking.

3.6 Parser implementation

The parser is implemented in Python as an LL(1) predictive parser based on the C-Pure grammar. Instead of relying on a set of mutually recursive procedures for every non-terminal, the implementation uses an explicit stack that controls the derivation process. This approach follows the predictive parsing table and makes the production selection process easier to trace.

The parsing process is driven by a stream of tokens generated by the lexical analyzer. Since the lexer produces token categories such as `Identifiers`, `Constants`, `Literals`, `Operators`, and `Punctuation`, the parser first maps those categories to the grammar symbols expected by the table. For example, identifiers are mapped to `id`, constants to `constant`, literals to `literal`, and operators or punctuation are mapped to their actual lexeme values.

The stack initially contains the start symbol `PROGRAM` and the end-of-input marker. During parsing, the top of the stack is compared with the current lookahead token:

- If the stack top is a terminal and matches the lookahead, the token is consumed.
- If the stack top is a non-terminal, the LL(1) table is consulted to select the corresponding production.
- If no production is available or a terminal mismatch occurs, the parser raises a syntactic error.

The parser also incorporates a semantic stack used to build AST nodes during the

derivation. In this way, the parser does not only verify the syntactic validity of the input, but also prepares the intermediate structure used by semantic analysis and code generation.

3.7 Abstract Syntax Tree (AST)

The Abstract Syntax Tree is the intermediate representation produced by the parser. While a derivation records which productions were applied, the AST captures the hierarchical structure of the program in a form that the semantic analyzer and the code generators can traverse efficiently. Unlike a full parse tree, the AST does not preserve every auxiliary symbol of the grammar; instead, it keeps the nodes that are relevant for meaning and later translation.

In this project, the AST works as the central connection between the front-end and the back-end of the compiler. The lexer and parser validate the input and construct the tree, the semantic analyzer traverses it to verify correctness, and the TAC generator later uses it to produce a linear intermediate representation.

3.7.1 Node structure

Each node is an instance of the `ASTNode` class. A node carries four main attributes:

- **node_type** (string): the syntactic category or role of the node, for example `PROGRAM`, `FUNCTION`, `ASSIGN`, `IF`, `WHILE`, `DO_WHILE`, `FOR`, `RETURN`, `BIN_OP`, `UNARY`, `ID`, `CONST` or `LITERAL`. This attribute drives both semantic analysis and downstream code generation.
- **children** (list of `ASTNode`): the ordered list of child nodes. The order follows the left-to-right appearance of the source program, preserving the information needed to reconstruct statements, expressions, and control-flow structures.
- **value** (string, optional): the lexeme, identifier, constant, literal, label, or operator associated with the node when applicable. For example, `x` may be stored as the value of an identifier node, and `+` may be stored as the value of a binary operation.
- **inferred_type** (string, optional): the data type computed or associated with the node. This attribute is used as a bridge between syntax and semantics, since type information can be consulted during assignments, expressions, function calls and return validation.

The class also provides a readable string representation of the tree, which is useful for debugging and for checking whether the parser has preserved the hierarchical structure of the source code.

3.7.2 Construction during parsing

AST construction is interleaved with the parsing process. The predictive parser uses a control stack to apply grammar productions and a semantic stack to build nodes when meaningful structures are recognized. As terminals and non-terminals are processed, leaf nodes and internal nodes are created and combined into larger subtrees.

Auxiliary grammar symbols, especially those used only to preserve LL(1) compatibility, do not necessarily become visible nodes in the final AST. This keeps the tree compact and readable. For example, expression helper symbols such as `TERM_PRIME`, `FACTOR_PRIME`, or `LOGIC_OR_PRIME` are useful for parsing, but their meaning is usually condensed into expression or operator nodes.

3.7.3 Role in semantic analysis

The semantic analyzer uses the AST as the structure to be traversed. Declaration nodes allow symbols to be inserted into the symbol table, identifier nodes allow lookup operations, expression nodes allow type compatibility checks, and control-flow nodes allow the analyzer to validate conditions and nested scopes.

For this reason, the AST is not only an output of the syntactic phase. It is the data structure that allows the compiler to connect grammatical recognition with semantic validation.

3.7.4 Role in code generation

After the program has been semantically validated, the AST becomes the input for the code-generation stage. The TAC generator traverses the tree and emits intermediate instructions according to the node type being visited. Assignment nodes become TAC assignments, expression nodes generate temporaries, conditional nodes generate labels and conditional jumps, and loop nodes generate entry and exit labels.

Therefore, the AST acts as the main intermediate representation of the compiler: it is compact enough to be processed efficiently, but expressive enough to preserve the relevant structure of the original program.

Listing 10: Conceptual AST structure

```
PROGRAM
|-- FUNCTION : main <int>
  |-- LOCAL_VAR : x <int>
  |-- LOCAL_VAR : y <int>
  |-- IF
    |-- BIN_OP : >
    |-- ASSIGN : x
```



```
| -- RETURN
    | -- ID : x
```

3.8 Semantic Analyzer

The semantic analyzer verifies that a syntactically valid program also satisfies the semantic rules of the C-Pure language. While the parser determines whether the token stream conforms to the grammar, the semantic analyzer verifies that the resulting structure is meaningful according to declarations, types, scopes, functions, arguments, and control-flow constraints.

The semantic validation is based on Syntax-Directed Translation (SDT). These rules associate semantic actions with grammar productions in order to compute attributes, manage identifiers, validate types and report semantic inconsistencies. In this implementation, semantic validation is supported mainly by the Symbol Table, the Function Table, and the traversal of the AST.

3.8.1 Symbol Table

The Symbol Table is a fundamental component of the semantic analysis phase and is responsible for managing declared identifiers in the program, including variables, functions, and function parameters. It is implemented as a scoped data structure that allows the compiler to resolve identifiers depending on their visibility and lifetime within the program.

In this implementation, the symbol table is designed as a stack of dictionaries, where each dictionary represents a different scope. When a new block is entered, such as a function body, a conditional block, or a loop structure, a new scope is pushed onto the stack using `enter_scope()` or loop-specific scope handling. When the block is exited, the corresponding scope is removed using `exit_scope()`. This mechanism ensures proper handling of nested scopes and prevents identifiers from being accessed outside their valid context.

Each entry in the symbol table stores essential information about an identifier, including its name, type, declaration line, category, initialization state, and whether it represents a function. For function symbols, the table can also store parameter information, which is later used to validate function calls and argument compatibility.

The main responsibilities of the symbol table in this system are the following:

- **Declaration management:** it stores identifiers when they are declared and prevents redeclaration within the same scope.
- **Scope control:** it manages global and local scopes using a stack-based structure, allowing nested blocks to be analyzed correctly.

- **Identifier lookup:** it searches for identifiers from the current scope back to the global scope, ensuring that variables and functions are declared before use.
- **Initialization tracking:** it records whether a variable has already been initialized before being used in expressions or assignments.
- **Function validation:** it stores function-related information, such as return type and parameters, to support validation of function calls.
- **Loop context support:** it supports the analysis of loop structures by managing scopes associated with `while`, `do-while`, and `for` statements.

This structure allows the semantic analyzer to detect errors such as undeclared identifiers, duplicate declarations, invalid use of `void` variables, incorrect function calls, and type incompatibilities during assignments, expressions, and return statements.

3.8.2 Semantic notation

The semantic rules are described using the following notation:

- `FT`: global function table.
- `ST`: symbol table of the current scope.
- `lookup(x)`: searches identifier `x` in the active scopes.
- `lookupCurrent(x)`: searches identifier `x` only in the current scope.
- `insert(x,t)`: inserts identifier `x` with type `t`.
- `insertParam(x,t)`: inserts a function parameter into the current function scope.
- `insertFunc(f,t,params)`: registers function `f` with return type `t` and parameter type list `params`.
- `arity(f)`: returns the number of parameters of function `f`.
- `paramTypes(f)`: returns the expected parameter types of function `f`.
- `numeric(t)`: true when `t` belongs to the numeric types accepted by the language.
- `compatible(t1,t2)`: verifies type compatibility between two types.
- `currentFuncType`: stores the return type of the function currently being analyzed.
- `insideLoop`: indicates whether the current node is being analyzed inside a loop.
- `type(X)`: represents the semantic type of symbol or expression `X`.
- `ok(X)`: indicates whether the derivation of `X` is semantically valid.

3.8.3 Type compatibility

The semantic specification defines the following compatibility rules:

- `compatible(int,int) = true`
- `compatible(int,char) = true`
- `compatible(char,int) = true`
- `compatible(char,char) = true`
- `compatible(void,*) = false`
- `compatible(*,void) = false`
- `compatible(string,*) = false`, except for specific output-oriented functions such as `printf`.

Literal conventions are also established. Numeric constants are treated as `int`; literals enclosed in single quotes are treated as `char`; and literals enclosed in double quotes are treated as `string`.

3.8.4 Validation procedure

The semantic analyzer follows a traversal procedure over the AST. Each visited node is interpreted according to its role in the language:

1. When a declaration node is found, the analyzer attempts to insert the identifier into the current symbol table.
2. When an identifier is used, the analyzer searches for it using `lookup(x)` from the current scope to the global scope.
3. When an assignment is found, the analyzer obtains the type of the left-side identifier and the type of the expression, then applies `compatible(t1,t2)`.
4. When a function declaration is found, the analyzer stores the function return type and its parameter list in the function table.
5. When a function call is found, the analyzer verifies that the function exists and that the number of arguments matches the expected arity.
6. When a `return` statement is found, the analyzer compares the returned expression type with `currentFuncType`.
7. When a control-flow structure is found, the analyzer validates the condition and creates the appropriate semantic context for nested statements.

8. When a loop structure is found, the analyzer enables `insideLoop` and handles the corresponding scope until the loop body finishes.

3.8.5 Main semantic validations

The SDT rules are applied to the main grammar constructs as follows:

- **Program and global declarations:** each global declaration is validated and inserted into the corresponding symbol or function table.
- **Type rules:** productions for `int`, `float`, `double`, `char`, and `void` synthesize the semantic type of the declaration.
- **Function definitions:** function declarations are inserted into the function table using their return type and parameter list. Parameters are inserted into the function scope.
- **Variable declarations:** local and global variables are inserted into the current symbol table. Redclarations in the same scope are reported as semantic errors.
- **Assignments:** the analyzer verifies that the assigned expression is compatible with the declared type of the identifier.
- **Function calls:** the analyzer verifies that the called function exists and that the number of supplied arguments matches the expected arity.
- **Return statements:** the type of the returned expression is compared with `currentFuncType`. A mismatch is reported as a semantic error.
- **Conditional statements:** the expression inside an `if` statement is validated as a condition, and the associated statement blocks are analyzed recursively.
- **Loop statements:** `while`, `do-while`, and `for` statements are analyzed with loop context enabled, allowing loop-specific semantic validation.
- **Increment and decrement:** the operators `++` and `--` are only valid over numeric identifiers.
- **Expressions:** arithmetic, relational, equality, and logical expressions propagate semantic types according to the grammar hierarchy.

3.8.6 Representative semantic rules

Listing 11: Representative SDT semantic rules

```
Program → Global Program
{ ok(Program) = ok(Global) and ok(Program1) }
```

```
Program → epsilon
{ ok(Program) = true }

Global → Type id GlobalRest
{ insert(id.lexeme, type(Type)); ok(Global) = ok(GlobalRest)
  }

GlobalRest → ( Params ) { StmtList }
{ insertFunc(id.lexeme, currentFuncType, paramTypes) }

Params → Type id ParamsRest
{ insertParam(id.lexeme, type(Type)) }

OptAssign → = Expr
{ if not compatible(id.type, type(Expr))
  then semantic error: incompatible initialization }

Stmt → return OptExpr ;
{ if not compatible(currentFuncType, type(OptExpr))
  then semantic error: return mismatch }

Stmt → Type id OptAssign ;
{ insert(id.lexeme, type(Type)); ok(Stmt) = ok(OptAssign) }

Stmt → id StmtIdRest
{ if lookup(id.lexeme) = null
  then semantic error: not declared }

StmtIdRest → = Expr ;
{ if not compatible(id.type, type(Expr))
  then semantic error: incompatible assignment }

StmtIdRest → ( Args ) ;
{ if arity(id.lexeme) != ArgCount
  then semantic error: invalid number of arguments }

StmtIdRest → ++ ;
{ if not numeric(type(id))
  then semantic error: invalid type for increment }
```

```
StmtIdRest → -- ;
{ if not numeric(type(id))
  then semantic error: invalid type for decrement }

IfStmt → if ( Expr ) { StmtList } ElsePart
{ if type(Expr) is not valid as a condition
  then semantic error: invalid condition }

WhileStmt → while ( Expr ) { StmtList }
{ insideLoop = true; ok(WhileStmt) = ok(Expr) and ok(
  StmtList) }

DoWhileStmt → do { StmtList } while ( Expr ) ;
{ insideLoop = true; ok(DoWhileStmt) = ok(StmtList) and ok(
  Expr) }

ForStmt → for ( ForInit Expr ; ForUpd ) { StmtList }
{ insideLoop = true;
  ok(ForStmt) = ok(ForInit) and ok(Expr) and ok(ForUpd) and
  ok(StmtList) }

ForInit → Type id = Expr ;
{ insert(id.lexeme, type(Type)); ok(ForInit) = ok(Expr) }

ForUpdRest → ++
{ ok(ForUpdRest) = true }

ForUpdRest → --
{ ok(ForUpdRest) = true }

Primary → id
{ if lookup(id.lexeme) = null
  then semantic error: not declared }

Primary → constant
{ type(Primary) = int }

Primary → literal
{ type(Primary) = string }

Primary → ( Expr )
```

```
{ type(Primary) = type(Expr) }
```

3.8.7 Semantic error detection

The semantic analyzer reports errors when a syntactically valid construction violates the rules of the language. The most relevant cases are duplicate declarations, undeclared identifiers, invalid `void` variables, incompatible assignments, invalid function calls, invalid argument counts, return type mismatches, and invalid use of increment or decrement operators.

In summary, the semantic analyzer extends the parser by ensuring that each valid syntactic construction is also meaningful according to the rules of the C-Pure language. This stage provides the necessary semantic reliability before using the AST as the basis for code-generation phases.

3.9 Why Top-Down and not Bottom-Up

Both Top-Down and Bottom-Up parsing are valid techniques for syntactic analysis. A Bottom-Up parser is powerful and can handle a larger class of grammars, but it usually requires more complex table construction and reduction mechanisms. In contrast, the C-Pure grammar was intentionally designed to be LL(1), which makes it suitable for a Top-Down predictive approach.

A Top-Down LL(1) parser was selected because its behavior is easier to connect with the formal grammar, FIRST sets, FOLLOW sets and parsing table. Since each parsing decision is made using one lookahead token, the process is deterministic and transparent. This also simplifies the integration of semantic stack actions and AST construction.

For this project, the Top-Down approach provides a clear balance between theoretical correctness and implementation simplicity. It allows the system to remain close to the grammar studied in class while still supporting semantic validation and later code-generation phases.

3.10 Three-Address Code Generation

Three-Address Code (TAC) is the intermediate representation produced after the AST has been syntactically and semantically validated. Its purpose is to transform the hierarchical structure of the AST into a linear sequence of simpler instructions. This makes the program easier to translate because each TAC instruction represents a small and explicit operation.

In TAC, each instruction usually contains at most one operator. Complex expressions are decomposed using temporary variables, while control-flow structures are represented

using labels and jumps. For this reason, TAC works as a bridge between the high-level source language and the final assembler output.

3.10.1 Input of the TAC generator

The input of this stage is the validated AST. At this point, the lexer has already recognized the tokens, the parser has verified the syntactic structure, and the semantic analyzer has confirmed that the program satisfies the semantic rules of the language. Therefore, the TAC generator can assume that the AST represents a valid C-Pure program.

The generator traverses the AST recursively and selects a translation strategy according to the node type being processed. For example, declaration and assignment nodes produce assignment instructions; expression nodes produce temporary variables; conditional nodes produce labels and conditional jumps; and loop nodes produce repeated control-flow patterns.

3.10.2 Generation procedure

The TAC generation process follows these general steps:

1. **Initialize instruction storage:** the generator begins with an empty list of TAC instructions.
2. **Initialize counters:** temporary variables and labels are generated using counters such as `t1`, `t2`, `L1`, and `L2`.
3. **Traverse the AST:** the generator visits each node in the AST and determines whether it represents an assignment, expression, return statement, conditional statement, or loop.
4. **Emit instructions:** when a node is translated, the corresponding TAC instruction is appended to the instruction list.
5. **Propagate results:** expression visitors return the temporary variable or value where the result of the expression was stored.
6. **Return the instruction list:** once the traversal finishes, the generated TAC list becomes the input for the assembler generator.

This design separates the translation process into small operations and avoids generating assembler instructions directly from the AST. Instead, TAC provides an intermediate layer that is easier to inspect, debug and translate.

3.10.3 Temporary variables

Temporary variables are used to store intermediate expression results. When an expression contains an operation, the generator creates a new temporary name and emits an instruction that assigns the result of that operation to the temporary.

For example, the expression $x + 1$ can be translated into:

```
t1 = x + 1
```

The temporary `t1` can then be used by another instruction, such as an assignment:

```
x = t1
```

This approach is useful because it decomposes complex expressions into independent steps and makes the order of evaluation explicit.

3.10.4 Labels and jumps

Labels are used to represent possible destinations in the control flow. Conditional and loop structures cannot be represented as a simple sequence of assignments, so they are translated using labels and jump instructions.

The generator creates labels using a label counter. A label such as `L1` or `L2` marks a specific position in the instruction list. Conditional jumps use these labels to decide where execution should continue when a condition is false or when a block has finished.

3.10.5 Translation of assignments

Assignments are translated by evaluating the right-hand expression and then assigning the resulting value to the identifier on the left. If the right-hand side is a simple value, the assignment can be emitted directly. If the right-hand side is an expression, the expression is first stored in a temporary variable.

Listing 12: TAC translation of assignments

```
x = 10

t1 = x + 1
x = t1
```

3.10.6 Translation of arithmetic expressions

Arithmetic expressions are translated by isolating one operation per instruction. This includes operations such as addition, subtraction, multiplication, division and modulo. The generated temporary variables preserve the intermediate results that may be used by later instructions.

Listing 13: TAC translation of an arithmetic expression

```
t1 = a * b
t2 = t1 + c
result = t2
```

3.10.7 Translation of return statements

A return statement is emitted as a TAC instruction containing the expression or value being returned. If the returned expression is complex, it is first evaluated into a temporary variable and then returned.

Listing 14: TAC translation of a return statement

```
t1 = x + 1
return t1
```

3.10.8 Translation of conditional statements

Conditional statements are translated by evaluating the condition and using a conditional jump to skip the **then** block when the condition is false. If an **else** branch exists, an additional jump is used to skip the **else** block after the **then** block has executed.

The general structure is the following:

Listing 15: General TAC pattern for if-else

```
t1 = condition
ifFalse t1 goto L1
    TAC for then block
goto L2
Label L1:
    TAC for else block
Label L2:
```

In this representation, L1 marks the beginning of the alternative branch and L2 marks the end of the entire conditional structure.

3.10.9 Translation of loop structures

Loop structures are translated by creating an entry label and an exit label. The entry label marks the point where the condition is evaluated, while the exit label marks the point where execution continues after the loop finishes.

A **while** statement can be represented as follows:

Listing 16: General TAC pattern for while

```
Label L1:
t1 = condition
ifFalse t1 goto L2
    TAC for loop body
goto L1
Label L2:
```

A `for` statement follows the same logic, but also includes initialization and update instructions. The initialization is emitted before the loop entry label, the condition is evaluated at the beginning of each iteration, and the update is emitted before jumping back to the entry label.

3.10.10 Representative TAC output

The following TAC fragment illustrates how assignments, conditions and return statements can be represented:

Listing 17: Representative TAC structure

```
x = 10
y = 5
t1 = x > y
ifFalse t1 goto L1
t2 = x + 1
x = t2
goto L2
Label L1:
Label L2:
return x
```

This output shows the main purpose of TAC: the original hierarchical program is converted into a linear sequence of explicit instructions. Temporaries represent intermediate expression results, and labels represent control-flow destinations.

3.10.11 Importance of TAC in the compiler

TAC is one of the most important stages of the final project because it connects the analysis phases with the code-generation phase. Without TAC, the assembler generator would need to translate directly from the AST, making the implementation harder to organize and extend. By introducing TAC, the compiler obtains a simpler intermediate form that can later be mapped into low-level instructions.

3.11 Assembler Code Generation

The assembler generator receives the TAC instruction list and translates it into a low-level representation inspired by x86-style assembly. This stage demonstrates how intermediate code can be lowered into instructions that resemble processor operations. It is the final translation stage of the compiler pipeline.

The generated output is not intended to be a complete executable binary. Instead, it is an educational assembler representation that shows how high-level source code can be progressively transformed into lower-level instructions.

3.11.1 Input of the assembler generator

The input of the assembler generator is the list of TAC instructions produced by the previous stage. Each TAC instruction is analyzed as a pattern. Depending on the pattern detected, the generator emits one or more assembler instructions.

For example, a TAC assignment such as:

```
x = 10
```

can be translated into:

```
mov eax, 10
mov x,  eax
```

The register `eax` is used as a working register to hold values temporarily before storing them in variables.

3.11.2 Assembler output structure

The assembler output is organized into two main sections:

- **Data section:** represents the area where global variables or reserved storage can be declared.
- **Text section:** contains the generated instructions, the global entry point and the body of `main`.

Listing 18: General assembler structure

```
.section .data
    # Global variables here

.section .text
.global main
main:
```

```
# Generated instructions
```

This structure makes the output resemble a conventional assembly program while keeping the implementation manageable for the scope of the project.

3.11.3 Translation procedure

The translation procedure follows these general steps:

1. **Read a TAC instruction:** the generator takes one instruction from the TAC list.
2. **Split the instruction into components:** the instruction is separated into tokens such as destination, operator, operands, labels or jump targets.
3. **Detect the TAC pattern:** the generator determines whether the instruction is an assignment, arithmetic operation, label, conditional jump, unconditional jump or return statement.
4. **Emit assembler instructions:** one or more low-level instructions are appended to the text section.
5. **Repeat until the TAC list ends:** the process continues until every TAC instruction has been translated.
6. **Merge sections:** the data section and text section are joined to produce the final assembler output.

3.11.4 Translation of direct assignments

Direct assignments are translated using `mov`. The value is first moved into a working register and then stored in the destination variable.

Listing 19: Assembler translation of a direct assignment

```
# TAC
x = 10

# ASM output
mov eax, 10
mov x, eax
```

This two-step strategy keeps the mapping simple and consistent for constants, identifiers and temporary values.

3.11.5 Translation of arithmetic operations

Arithmetic operations are translated by loading the first operand into a register, applying the operation with the second operand, and then moving the result into the destination variable.

Listing 20: Assembler translation of an arithmetic operation

```
# TAC
t1 = x + 1

# ASM output
mov eax, x
add eax, 1
mov t1, eax
```

For the implemented subset, addition is the clearest representative case. The same strategy can be extended to subtraction, multiplication, division and modulo using the corresponding assembly instructions.

3.11.6 Translation of labels

Labels in TAC are preserved as destinations in the assembler output. A TAC label such as `Label L1:` becomes an assembler label:

Listing 21: Assembler translation of labels

```
# TAC
Label L1:

# ASM output
L1:
```

These labels allow conditional and unconditional jumps to refer to specific points in the generated code.

3.11.7 Translation of conditional jumps

Conditional jumps are produced from TAC instructions such as `ifFalse t1 goto L1`. The assembler generator translates this pattern by comparing the condition temporary with zero and jumping when the condition is false.

Listing 22: Assembler translation of a conditional jump

```
# TAC
ifFalse t1 goto L1
```

```
# ASM output
cmp t1, 0
je L1
```

The instruction `cmp` compares the condition result against zero. The instruction `je` jumps to the target label when the comparison indicates that the condition is false.

3.11.8 Translation of unconditional jumps

Unconditional jumps are translated directly with `jmp`. These instructions are used to skip alternative branches or return to the beginning of a loop.

Listing 23: Assembler translation of an unconditional jump

```
# TAC
goto L2

# ASM output
jmp L2
```

3.11.9 Translation of return statements

Return statements place the resulting value in the expected output register and then emit a return instruction. In this project, `eax` is used as the conventional return register.

Listing 24: Assembler translation of a return statement

```
# TAC
return x

# ASM output
mov eax, x
ret
```

3.11.10 Representative assembler output

Listing 25: Representative assembler output

```
.section .data
    # Global variables here

.section .text
```

```
.global main
main:
    mov eax, 10
    mov x, eax
    mov eax, 5
    mov y, eax
    cmp t1, 0
    je L1
    mov eax, x
    add eax, 1
    mov x, eax
    jmp L2
L1:
L2:
    mov eax, x
    ret
```

3.11.11 Importance of assembler generation in the project

This stage is one of the most important components of the final compiler because it demonstrates the transition from intermediate representation to low-level output. The assembler generator completes the compiler pipeline by showing how validated source code can be progressively transformed into instructions that resemble processor-level operations.

It also confirms the usefulness of the previous phases. The lexer, parser, AST and semantic analyzer prepare the program so that code generation can be performed over a valid and structured representation. TAC simplifies the program, and the assembler generator maps those simplified instructions into a lower-level form.

3.12 Use-Case Based GUI Validation

The validation of the graphical interface was documented through use cases. Each use case describes a realistic interaction between the user and the C-Pure Compiler IDE. The purpose is not to test isolated buttons, but to show how the interface behaves when the compiler receives different kinds of input programs.

For each case, a screenshot of the compiler interface can be included to visually support the described behavior. The screenshots should show the source code entered in the editor, the active output tab, and the state of the pipeline badges.

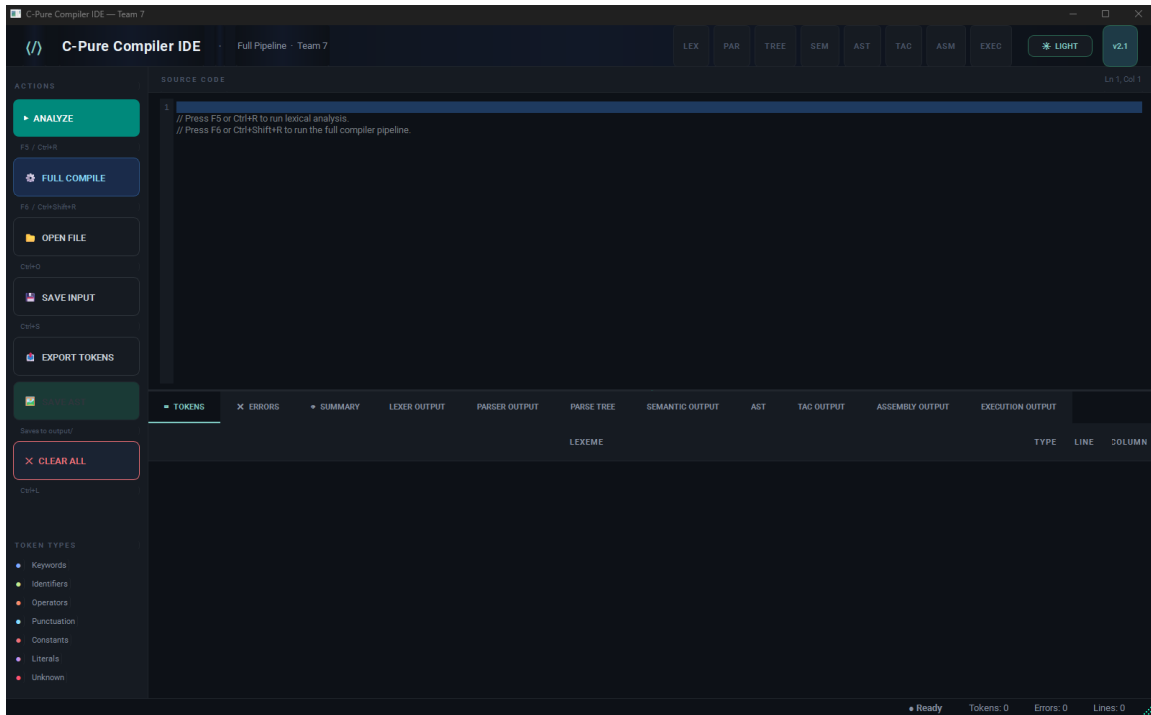


Figure 1: GUI running

Use-Case Criterion

In this section, the GUI is treated as the user-facing environment of the compiler. Therefore, each use case describes what the user does, what the compiler processes, and how the interface communicates the result through tabs, tables, badges, messages, and exported outputs.

3.12.1 Use Case 1: Running Lexical Analysis

Objective. Verify that the user can execute lexical analysis from the GUI and inspect the tokens recognized by the compiler.

User action. The user writes or loads a C-Pure source program in the editor and presses the **Analyze** button, or uses the shortcut **F5 / Ctrl+R**.

Interface behavior. The GUI sends the source code to the lexer and updates the lexical-analysis section without executing the complete compiler pipeline. The **LEX** badge changes from idle to running and then to either success or error depending on the result. The token table is populated with the recognized lexemes, their categories, and their positions in the source code.

The **Tokens** tab displays the lexical output in a structured table with the columns **Lexeme**, **Type**, **Line**, and **Column**. At the same time, the summary panel updates the total number of tokens, lexical errors, analyzed lines, and token distribution by

category.

```

—— Lexer Output Report ——

Total tokens : 5
Lexical errors : 0
Lines scanned : 1

—— Token Distribution ——

Identifiers    2 token(s)
Operators      1 token(s)
Constants      1 token(s)
Punctuation    1 token(s)

—— First 50 Tokens ——

[ 1:1 ] Identifiers  int
[ 1:5 ] Identifiers  i
[ 1:7 ] Operators   =
[ 1:9 ] Constants   1
[ 1:10] Punctuation ;

```

Figure 2: GUI behavior after running lexical analysis. The token table is populated and the lexical summary is updated.

Expected result. If the input contains only valid symbols, the interface marks lexical analysis as successful. If an invalid symbol is found, the interface reports it as an **Unknown** token and displays the corresponding line and column in the errors tab.

3.12.2 Use Case 2: Running the Full Compilation Pipeline

Objective. Verify that the user can compile a valid C-Pure program and inspect all intermediate compiler outputs.

User action. The user writes a valid source program and presses the **Full Compile** button, or uses the shortcut **F6 / Ctrl+Shift+R**.

Listing 26: Example input for a successful compilation use case

```

int main() {
    int x = 10;
    int y = 20;
    int z = x + y;
    return z;
}

```

Interface behavior. The GUI executes the complete compiler pipeline. While the process is running, the pipeline badges indicate progress through the different stages: lexical analysis, parsing, parse tree construction, semantic analysis, AST visualization, TAC generation, assembly generation, and execution simulation.

When the program is valid, each badge finishes in the success state. The interface enables the corresponding output tabs and displays the result of each compiler stage. The **Summary** tab reports that no lexical errors were detected, the **AST** tab shows the abstract syntax tree, and the later tabs show TAC, assembly, and execution information.

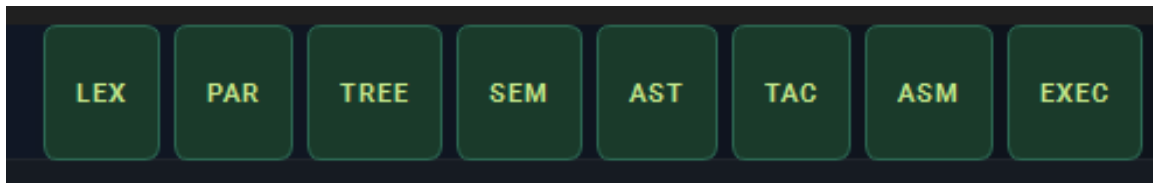


Figure 3: GUI behavior after compiling a valid C-Pure program. The pipeline badges indicate successful completion of all stages.

Expected result. The interface should display a complete successful pipeline. The final status bar should indicate that the pipeline was completed, and the output tabs should contain the generated intermediate representations.

3.12.3 Use Case 3: Syntax Error During Compilation

Objective. Verify that the GUI reports syntax errors clearly and blocks the compiler stages that depend on a valid parse tree.

User action. The user enters a program with a syntactic mistake and runs the full compilation pipeline.

Listing 27: Example input for a syntax-error use case

```
int main() {
    int x = 10
    return x;
}
```

Interface behavior. The lexer is able to recognize the individual tokens, but the parser rejects the sequence because the declaration is missing a semicolon. The GUI displays the parsing error in the **Errors** tab and marks the affected pipeline stages as failed.

Since the parse tree cannot be built correctly, the interface does not continue with semantic analysis, AST generation, TAC generation, assembly generation, or execu-

tion simulation. Instead, those tabs show explanatory messages indicating that the corresponding output was not generated due to parsing errors.



Figure 4: GUI behavior when the compiler detects a syntax error. The interface reports the parsing problem and stops dependent stages.

Expected result. The GUI should guide the user to the syntax error instead of producing incomplete or invalid compiler outputs. The relevant error message should identify the syntactic problem and the pipeline should stop safely.

3.12.4 Use Case 4: Semantic Error During Compilation

Objective. Verify that the GUI distinguishes between syntactically valid programs and semantically invalid programs.

User action. The user enters a program that follows the grammar but contains a semantic error, then runs the full compilation pipeline.

Listing 28: Example input for a semantic-error use case

```
int main() {
    int x = 10;
    y = x + 1;
    return x;
}
```

Interface behavior. The lexical and syntactic stages complete successfully because the tokens are valid and the structure of the program matches the grammar. However, the semantic analyzer detects that the identifier `y` is used without being declared first. The GUI reports the semantic error in the **Errors** tab and in the semantic-output tab. The semantic badge is marked as failed, and the stages that depend on a semantically valid AST are blocked. As a consequence, TAC, assembly, and execution simulation are not generated.

Expected result. The interface should make clear that the program is syntactically correct but semantically invalid. The user should be able to identify the semantic cause of the failure directly from the error message.

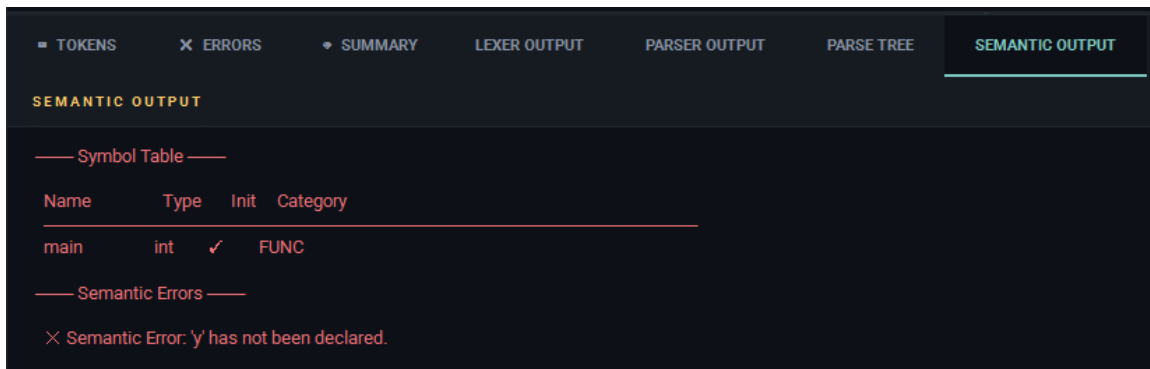


Figure 5: GUI behavior when the semantic analyzer detects an undeclared variable. The interface shows the semantic error and prevents later code generation stages.

3.12.5 Use Case 5: Inspecting and Saving the AST

Objective. Verify that the user can inspect the generated AST and save it for documentation or debugging purposes.

User action. The user compiles a valid program and opens the AST tab. If the AST was generated correctly, the user selects the **Save AST** option.

Interface behavior. When compilation succeeds, the GUI displays the AST as an interactive tree. If Graphviz is available in the environment, the interface also renders the AST as a graphical image. The **Save AST** button becomes available only when there is a valid AST output.

If the graphical AST image exists, it is saved as an image file. If the Graphviz image cannot be generated, the GUI still allows the user to save the tree representation as a text file.

Expected result. The interface should not enable AST export before a valid AST exists. Once the AST is available, the user should be able to inspect it and save it without rerunning the compiler manually.

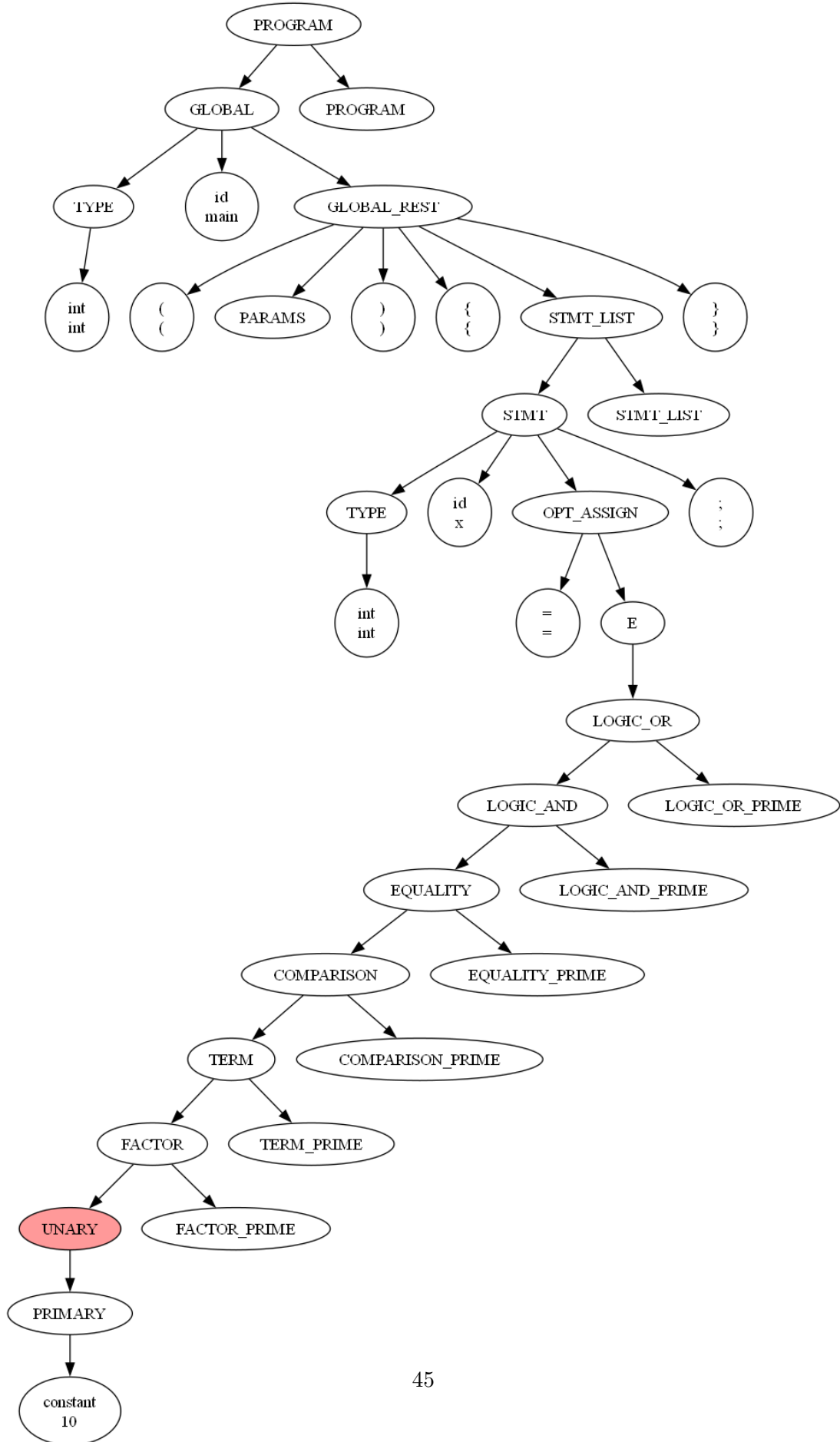
3.12.6 Use Case 6: Clearing the Workspace

Objective. Verify that the user can reset the interface after running one or more compilation attempts.

User action. The user presses the **Clear All** button, or uses the shortcut **Ctrl+L**.

Interface behavior. The GUI clears the editor content, token table, error messages, summary cards, output tabs, AST information, and pipeline badges. The status bar returns to the initial ready state, and the counters for tokens, errors, and lines are reset.

Expected result. The GUI should return to a clean state, allowing the user to start a new analysis without keeping previous outputs or error messages.



3.12.7 Use-Case Summary

Use case	User action	Expected interface behavior
Lexical analysis	Press Analyze	Fill token table, update summary, and report lexical errors if present.
Full compilation	Press Full Compile	Execute all compiler stages and update the pipeline badges and output tabs.
Syntax error	Compile syntactically invalid code	Show parsing error and block dependent stages.
Semantic error	Compile semantically invalid code	Show semantic error and prevent TAC, assembly, and execution generation.
AST inspection	Open AST tab after successful compilation	Display the AST and enable AST saving.
Clear workspace	Press Clear All	Reset editor, outputs, counters, and pipeline badges.

References

- [1] C. Staff, “What Is Lexical Analysis?,” Coursera, Apr. 10, 2024. [Online]. Available: <https://www.coursera.org/articles/lexical-analysis>
- [2] “Introduction of Lexical Analysis,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>
- [3] “Program to detect tokens in a C code,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/c/c-program-detect-tokens-c-program/>
- [4] “C Tokens,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/es-es/cpp/c-language/c-tokens?view=msvc-170>
- [5] GeeksforGeeks, “Lexical Error,” GeeksforGeeks, Jul. 23, 2025. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/lexical-error/>
- [6] GeeksforGeeks, “Regex Tutorial - How to write Regular Expressions,” GeeksforGeeks, Dec. 22, 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/write-regular-expressions/>

-
- [7] I. Vladimir, “Finite Automata,” *Lenguajes Formales y Autómatas*. [Online]. Available: https://ivanvladimir.gitlab.io/lfya_book/docs/02lam%C3%A1quinasinmemoria/04aut%C3%B3matafinito/. [Accessed: May 28, 2026].
 - [8] “Types of Parsers in Compiler Design,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/types-of-parsers-in-compiler-design/>
 - [9] “Classification of Top Down Parsers,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/classification-of-top-down-parsers/>
 - [10] “FIRST and FOLLOW in Compiler Design,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/why-first-and-follow-in-compiler-design/>
 - [11] “Syntax Directed Translation in Compiler Design,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/syntax-directed-translation-in-compiler-design/>
 - [12] GeeksforGeeks, “Three Address Code in Compiler,” GeeksforGeeks, Apr. 11, 2026. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/three-address-code-compiler/>.
 - [13] AcademiaLab, “Assembly Language,” *AcademiaLab*. [Online]. Available: <https://academia-lab.com/enciclopedia/lenguaje-ensamblador/>.